NETWORK TRAFFIC ANALYZER AND VISUALIZER USING PYTHON

Project Report

Student Name	Kurhade Sarang Keshav
Roll Number	2025BCS0009
Course / Batch	ITW III / Batch 2
Institute	INDIAN INSTITUE OF INFORMATION TECHNOLOGY KOTTAYAM
Faculty / Guide	Dr . Victor Paul

An educational project demonstrating data cleaning, statistical analysis, anomaly flagging, and network visualization.

Abstract

This project presents a Python-based Network Traffic Analyzer and Visualizer that processes network traffic records stored in a CSV file. It validates and cleans the data, summarizes packet and byte counts, compares traffic by protocol, identifies active source and destination IP addresses, and builds a directed graph of communication between IP addresses. It also applies a statistical threshold to flag records with unusually high packet counts or byte volumes. Results are displayed through charts and exported to a new CSV file. The current implementation uses previously collected or sample data; it does not capture live traffic.

Keywords: Python, Pandas, NetworkX, Matplotlib, network traffic, data analysis, anomaly detection.

1. Introduction

Computer networks generate traffic whenever devices exchange data. Raw traffic records can be difficult to interpret without processing and visualization. A network traffic analyzer organizes records into summaries and visual patterns to help users understand communication activity.

This project uses Python libraries to transform a traffic CSV file into readable statistics, graphs, and a list of potentially unusual records. It is an educational prototype, not a production-grade intrusion detection system.

2. Problem Statement

Given a CSV dataset containing source IP, destination IP, protocol, packet count, byte count, and timestamp, develop a program that validates the dataset, summarizes network activity, visualizes communication between IP addresses, compares traffic across protocols, and flags unusually large traffic records for review.

3. Objectives

• Load network traffic records from a CSV file.

• Validate required columns and remove incomplete or invalid records.

• Calculate record count, unique IP addresses, total packets, and transferred bytes.

• Find source and destination IP addresses with the largest byte totals.

• Compare packet counts and byte volumes across protocols.

• Create a directed communication graph using NetworkX.

• Flag potential anomalies using a statistical threshold.

• Export cleaned data, including anomaly flags, to a CSV file.

4. Tools and Technologies

Technology	Purpose
Python	Main programming language.
Pandas	Reads CSV files, cleans records, groups data, and calculates summaries.
NumPy	Imported in the current script but not used; remove it or add numerical operations.
NetworkX	Creates the directed communication graph.
Matplotlib	Draws the network graph, bar charts, and histogram.
CSV dataset	Stores IP addresses, protocol, packets, bytes, and timestamps.

5. Dataset Description

The program expects **network_traffic.csv** in the working directory. The required columns are:

Column	Description
source_ip	IP address sending the traffic record.
destination_ip	IP address receiving the traffic record.
protocol	Protocol label, such as TCP or UDP if present.

packets	Number of packets represented by the record.
bytes	Amount of data represented by the record, in bytes.
timestamp	Date and time associated with the record.

The actual results depend on the quality and meaning of the supplied dataset. The program does not generate the source data itself.

6. Methodology

Step 1 — Load and validate: Pandas reads the CSV. The program checks that all required columns exist and stops if any are missing.

Step 2 — Clean: Rows with missing required values are removed. Packet and byte values are converted to numbers, timestamps are parsed, and invalid or negative packet/byte values are excluded.

Step 3 — Summarize: The program calculates record count, unique IP count, total packets, total bytes in KB, protocol distribution, and the five source and destination IPs with the highest byte totals.

Step 4 — Flag unusual records: It uses the median and median absolute deviation (MAD) to set a threshold. If MAD is zero, it uses the 95th percentile. A row is flagged if its packet count or byte count exceeds its corresponding threshold.

Step 5 — Build graph: Each IP becomes a node. Each source-to-destination pair becomes a directed edge. Repeated pairs are combined by summing packets and bytes.

Step 6 — Visualize and export: Matplotlib displays the graph, protocol bar charts, and byte histogram. The cleaned dataset with an anomaly column is saved as **analyzed_network_traffic.csv**.

7. Graphs and Interpretation

7.1 Network Communication Graph

Nodes represent IP addresses. Arrows show communication direction from source to destination. Thicker edges indicate a greater total byte volume for that connection, scaled relative to the largest edge. The graph helps reveal high-volume communication links.

7.2 Packets by Protocol

The first bar chart sums packet counts for each protocol. The horizontal axis shows protocol labels and the vertical axis shows total packets. Taller bars indicate more packets associated with that protocol.

7.3 Bytes by Protocol

The second bar chart sums byte counts for each protocol. Taller bars indicate a greater total volume of data. This differs from packet count because packets can have different sizes.

7.4 Distribution of Traffic Volume

The histogram groups individual records into byte-size ranges. The horizontal axis represents bytes per record and the vertical axis represents the number of records in each range. The dashed line marks the byte threshold used by the anomaly rule.

8. Anomaly-Flagging Logic

The program uses a statistical heuristic, not a trained machine-learning model. A record is flagged if its packet count or byte count exceeds its threshold. A large transfer may be legitimate, and some attacks may not produce large values, so alerts are not proof of malicious activity.

9. Expected Output

- Console summary of traffic totals and protocol distribution.
- Top five source and destination IP addresses by transferred bytes.
- A table of records that exceed the anomaly thresholds, if any.
- A directed IP communication graph.
- Two protocol bar charts and one traffic-volume histogram.
- An output CSV named analyzed_network_traffic.csv with cleaned records and anomaly flags.

Actual numerical results are not included because the dataset and execution output were not provided. Insert screenshots and results from your own run before submission.

10. Limitations

- The program reads a CSV file and does not capture live packets.
- Analysis quality depends on the dataset's accuracy and representativeness.
- The anomaly rule may produce false positives or miss unusual activity.
- The graph can become crowded when many IP addresses are present.
- The program analyzes protocol labels as supplied and does not independently verify them.
- The current code does not include a Streamlit dashboard or database integration.

11. Future Enhancements

- Add a Streamlit dashboard with filters for IP, protocol, and time range.
- Add interactive Plotly charts.
- Add time-based traffic trends and comparisons.
- Improve anomaly detection with baselines per IP and protocol.
- Add optional SQLite storage for historical analysis.
- Implement a separate live packet-capture module if required, with suitable permissions and privacy safeguards.

12. Conclusion

The Network Traffic Analyzer and Visualizer demonstrates how Python can clean, summarize, and visualize network traffic data. Pandas supports data handling and aggregation, NetworkX represents communication between IP addresses, and Matplotlib displays the results. A statistical rule highlights potentially unusual

records for further review. The project can be extended with interactive dashboards, time-based analysis, and more advanced detection methods.

13. References

1. Pandas documentation — https://pandas.pydata.org/docs/
2. NetworkX documentation — https://networkx.org/documentation/stable/
3. Matplotlib documentation — https://matplotlib.org/stable/
4. Python documentation — https://docs.python.org/3/

Before submission: Fill in the cover-page details, confirm the report matches your final code, insert screenshots from your own run, and add actual results. Do not claim live capture or a Streamlit dashboard unless those features have been implemented and tested.